

Compiladores, 2025/2026

Trabalho prático, parte 1

– Linguagem Fava: expressões, consistência de tipos –

Fernando Lobo

1 Introdução

Neste e nos restantes trabalhos da disciplina iremos desenvolver um compilador para uma linguagem imperativa chamada Fava. Para começar, iremos nos restringir a fazer a compilação de programas que apenas permitem um tipo de instrução: `print`.

Nos restantes trabalhos iremos acrescentar outro tipo de instruções e outro tipo de funcionalidades à linguagem.

Caso o input não tenha erros, o vosso compilador deve gerar bytecodes para uma máquina virtual baseada num stack, que também deverá ser implementada por vós.

2 Sintaxe da linguagem Fava

Um programa em Fava é uma sequência de uma ou mais instruções, cada qual consistindo na palavra reservada `print`, seguido de uma expressão, seguido de um ponto-e-vírgula.

A linguagem Fava é *case insensitive*. Isto é, não faz distinção entre maiúsculas e minúsculas. Assim, é indiferente escrever `print`, `PRINT` ou `pRInt`.

A expressão pode envolver operadores aritméticos, lógicos, e também de concatenação de strings. A sintaxe de uma expressão pode ser descrita de forma indutiva através das seguintes regras.

1. um literal (um número inteiro, um número real, uma string, `true`, `false`) é por definição uma expressão.
2. um operador unário seguido de uma expressão, é uma expressão.

3. uma expressão, seguida de um operador binário, seguido de outra expressão, é também uma expressão.
4. Um parêntesis a abrir, seguido de uma expressão, seguido de um parêntesis a fechar, é também uma expressão.

A linguagem Fava suporta 4 tipos de dados: bool, integer, real, string. O tipo bool suporta os valores **true** e **false** (equivalente a boolean em Java). O tipo integer suporta inteiros (equivalente a um int em Java). O tipo real suporta números reais (equivalente a um double em Java). O tipo string suporta sequência de caracteres delimitados por aspas.

Tal como em C e Java, podemos ter comentários de uma só linha (começam com `//` e vão até ao final da linha) ou multi-linha (começam com `/*` e terminam com `*/`).

Para facilitar-vos a vida, dou-vos as regras lexicais em ANTLR que permitem ter este tipo de comentários.

```
SL_COMMENT :   '//' .*? (EOF|'\n') -> skip;   // single-line comment
ML_COMMENT :   '/*' .*? '*/' -> skip ;         // multi-line comment
```

Regras lexicais para literais integer e real

As regras lexicais para integer e real em Fava são idênticas às regras lexicais para os int e double em Java, respectivamente.

2.1 Exemplo de um programa em Fava

```
/*
    Exemplo de programa em Fava
*/
print 1 + 2 * 3;
PRINT 1 + 2.0 * 3;
print 7 mod (1 + 4);
Print true AND (5 < 3.14159);
print "pi = " || 3.14159;
print "ma" || "ria" = "maria";
```

2.2 Precedência dos operadores

A tabela seguinte apresenta a precedência dos operadores da linguagem Fava. Quanto mais acima na tabela, maior é a precedência do operador (e menor é o número que aparece na 1ª coluna da tabela). Dizer que um operador tem maior precedência que outro, significa que esse operador deve ser avaliado primeiro do que o operador com precedência mais baixa.

Os operadores que aparecem na mesma linha, têm o mesmo nível de precedência. Nesse caso, a ordem pela qual as operações são feitas é da esquerda para a direita, i.e. todos têm associatividade à esquerda.

Precedência	Descrição	Operadores
1	Parênteses	()
2	Unário	- not
3	Multiplicação e Divisão	* / mod
4	Soma e Subtração	+ -
5	Concatenação	
6	Relacional	< > <= >=
7	Igualdade e Desigualdade	= <>
8	E lógico	and
9	Ou lógico	or

Os operadores listados na tabela acima têm o significado usual e são em tudo semelhantes a operadores que existem no C e Java. Note porém que o símbolo usado para alguns deles é diferente, nomeadamente os operadores `not`, `mod`, `||`, `=`, `<>`, `and`, `or` (que em C e Java são escritos como `!`, `%`, `+`, `==`, `!=`, `&&`, `||`, respectivamente).

3 Regras sobre tipos de dados

Tal como acontece na maioria das linguagens de programação, a utilização dos operadores acima descritos obedecem a algumas regras, dependendo do tipo de dados do(s) operando(s). Por exemplo, a seguinte instrução,

```
print true - 6;
```

apesar de ser sintaticamente correcta, não é válida porque o operador binário `-` não pode ser aplicado quando um dos operandos é do tipo `bool`. Como tal, o compilador deverá emitir uma mensagem de erro apropriada neste tipo de situação. De seguida apresenta-se as regras sobre os tipos de dados da linguagem Fava.

3.1 Regras para os literais

- um literal integer é uma expressão do tipo integer
- um literal real é uma expressão do tipo real
- uma string delimitada por aspas é uma expressão do tipo string
- `true` é uma expressão do tipo bool
- `false` é uma expressão do tipo bool

3.2 Regras para operadores unários

<i>op</i>	<i>expr</i>	<i>op expr</i>
-	integer	integer
-	real	real
<code>not</code>	bool	bool

Cada uma das linhas da tabela especifica uma regra. Por exemplo, a 1^a linha da tabela acima diz que o operador unário - aplicado a uma expressão do tipo integer, dá origem a uma expressão do tipo integer. Como se pode constatar, o operador unário - só pode ser aplicado para expressões do tipo integer ou real. Por sua vez o operador unário `not` só pode ser aplicado para expressões do tipo bool.

3.3 Regras para operadores binários

<i>op</i>	<i>expr₁</i>	<i>expr₂</i>	<i>expr₁ op expr₂</i>
+ - * / mod	integer	integer	integer
+ - * /	integer	real	real
+ - * /	real	integer	real
+ - * /	real	real	real
 	bool	string	string
 	string	bool	string
 	integer	string	string
 	string	integer	string
 	real	string	string
 	string	real	string
 	string	string	string
and or	bool	bool	bool
< > <= >=	integer	integer	bool
< > <= >=	integer	real	bool
< > <= >=	real	integer	bool
< > <= >=	real	real	bool
= <>	bool	bool	bool
= <>	integer	integer	bool
= <>	integer	real	bool
= <>	real	integer	bool
= <>	real	real	bool
= <>	string	string	bool

Novamente, cada linha da tabela acima especifica uma regra. A aplicação de operadores fora do contexto acima descrito deverá originar um erro semântico de inconsistência de tipos (não de parsing), que deve ser reportado pelo compilador de forma apropriada.

Como referido anteriormente, o significado dos operadores é em tudo idêntico ao da linguagem Java. Note por exemplo que o operador **||** é usado para fazer concatenação de strings, quando pelo menos um dos operandos é uma string, sendo que o compilador deve gerar código para fazer conversões de tipos se necessário. Por exemplo, na expressão:

`"pi = " || 3.14159`

a expressão do lado esquerdo é uma string e a expressão do lado direito é um real. Como tal, é suposto o compilador gerar código para converter 3.14159 para a string "3.14159", e só depois gerar código para fazer a concatenação das strings.

4 Máquina Virtual F

O vosso compilador de Fava deverá gerar bytecodes para uma máquina virtual chamada F (versão mini). Trata-se de uma máquina virtual *stack-based*, e é uma extensão daquela que já vimos na aula teórica 13 e aula PL 6.

A especificação da máquina virtual F está disponível num ficheiro à parte disponível na tutoria: `FVM-mini.pdf`

5 Exemplos de inputs/outputs para o mooshak

Exemplo A

Input

```
print 1 + 2 * 3;
```

Output

```
*** Constant pool ***
*** Instructions ***
0: iconst 1
1: iconst 2
2: iconst 3
3: imult
4: iadd
5: iprint
6: halt
*** VM output ***
7
```

Exemplo B

Input

```
print 1 + 2 * 3.0;  
print "ola " || "maria";
```

Output

```
*** Constant pool ***  
0: 3.0  
1: "ola "  
2: "maria"  
*** Instructions ***  
0: iconst 1  
1: itod  
2: iconst 2  
3: itod  
4: dconst 0  
5: dmult  
6: dadd  
7: dprint  
8: sconst 1  
9: sconst 2  
10: sconcat  
11: sprint  
12: halt  
*** VM output ***  
7.0  
ola maria
```

Exemplo C

Input

```
/*  
  Exemplo de programa em Fava  
*/
```

```
print 1 + 2 * 3;
PRINT 1 + 2.0 * 3;
print 7 mod (1 + 4);
Print true AND (5 < 3.14159);
print "pi = " || 3.14159;
print "ma" || "ria" = "maria";
```

Output

```
*** Constant pool ***
0: 2.0
1: 3.14159
2: "pi = "
3: "ma"
4: "ria"
5: "maria"
*** Instructions ***
0: iconst 1
1: iconst 2
2: iconst 3
3: imult
4: iadd
5: iprint
6: iconst 1
7: itod
8: dconst 0
9: iconst 3
10: itod
11: dmult
12: dadd
13: dprint
14: iconst 7
15: iconst 1
16: iconst 4
17: iadd
18: imod
19: iprint
20: tconst
21: iconst 5
22: itod
23: dconst 1
24: dlt
```



```
25: and
26: bprint
27: sconst 2
28: dconst 1
29: dtos
30: sconcat
31: sprint
32: sconst 3
33: sconst 4
34: sconcat
35: sconst 5
36: seq
37: bprint
38: halt
*** VM output ***
7
7.0
2
false
pi = 3.14159
true
```

Exemplo D

Input

```
print 2 & + 5;
```

Output

```
Input has lexical errors
```

Exemplo E

Input

```
print azul 4;  
print verde 33;
```

Output

```
Input has lexical errors
```

Exemplo F

Input

```
print 4;;;
```

Output

```
Input has parsing errors
```

Exemplo G

Input

```
print "ola";  
print 3 * 2 + false;  
print true || 2.0 + 3;  
print "maria" mod 3.14;
```

Output

```
error in line 2: operator + is invalid between integer and bool
error in line 3: operator || is invalid between bool and real
error in line 4: operator mod is invalid between string and real
```

6 Recomendações

1. Começam por especificar a gramática em ANTLR e certifiquem-se que está correcta. Para tal devem testar vários inputs (com e sem erros gramaticais).
2. Após certificarem-se que a gramática está correcta, implementem a deteção de erros semânticos, que no contexto deste trabalho se resumem a erros de inconsistência de tipos. Para tal devem percorrer a árvore sintática e anotar os nós das expressões com informação sobre o seu tipo.
3. Após certificarem-se que não há erros de inconsistência de tipos, podem implementar o gerador de código. Para tal, devem percorrer novamente a árvore sintática e, usando a informação obtida no ponto anterior, gerar instruções adequadas para a máquina virtual.
4. Não basta testarem os exemplos ilustrados no enunciado. O vosso trabalho deve ser testado de forma exaustiva para garantir com o máximo de certeza possível que o vosso compilador funciona do modo esperado.

7 Requisitos

- O trabalho deve ser feito em Java usando o ANTLR 4, e submetido ao mooshak.
- Devem submeter um ficheiro zip que deverá conter uma pasta chamada **src** onde está todo o código que desenvolveram.
- A gramática deve chamar-se **Fava.g4** e deverá estar na pasta **src**
- A pasta **src** deverá conter o código gerado pelo ANTLR, supostamente numa pasta/package chamada **Fava**
- Nota importante: NÃO DEVEM incluir o ficheiro **antlr-4.13.2-complete.jar** na vosso zip. O servidor do mooshak já lá tem esse ficheiro.
- No código apenas pode haver um ficheiro java que tenha o método **main**. Não pode haver mais nenhum **main**, mesmo que esteja comentado. O nome do ficheiro que tem o método **main** deve chamar-se **FavaCompileAndRun.java**

- O vosso código é submetido ao mooshak, pelo que deverá poder ler o input a partir do *standard input*. Não obstante, o código deve estar também preparado para poder receber o input (nome do ficheiro Fava a compilar, e eventuais flags) a partir de argumentos passados à função `main`.
- O método `main` deverá definir as seguintes 2 flags que servem para controlar o modo como a emissão de erros é feita.

```
boolean showLexerErrors = false;
boolean showParserErrors = false;
```

Quando o código é submetido ao mooshak, as flags devem ter o valor `false` de modo a não mostrar as mensagens de erro. Porém, se o valor das flags for alterado para `true`, o vosso programa é suposto emitir os respectivos erros com mensagens apropriadas.

- Para efeitos de submissão ao mooshak:
 - se o input tiver erros lexicais, o programa deverá enviar a seguinte mensagem para o output e terminar: `Input tem erros lexicais`
 - se não tiver erros lexicais, mas tiver erros de parsing, o programa deverá enviar a seguinte mensagem para o output e terminar: `Input tem erros de parsing`
 - se não tiver erros lexicais nem de parsing, mas tiver erros de consistência de tipos, o programa deverá reportar esses erros. Cada mensagem de erro deve indicar a linha onde o erro ocorre, seguido de mensagem apropriada (ver exemplo G).
 - se não tiver erros lexicais, nem de parsing, nem de consistência de tipos, o programa deverá gerar bytecodes de acordo com a especificação da máquina virtual e guardá-los num ficheiro com o nome `bytecodes.bc`. Após ser gerado, esse ficheiro deverá ser lido de imediato e a máquina virtual deverá executar os bytecodes.

8 Sobre o Mooshak

- <http://deei-mooshak.ualg.pt/~flobo/>, concurso Comp26, problema A.
- Apenas um dos elementos do grupo se deve registar no mooshak, devendo partilhar a password com o(a) colega de grupo. Ao fazer o registo o nome do User deve ser forçosamente o nome do vosso grupo como está na tutoria, algo do género PL3g12. Para o Email usem o vosso email da ualg. Registos que não cumpram esta regra serão ignorados e/ou eliminados.

- Não use caracteres acentuados no código, nem mesmo nos comentários, uma vez que o mooshak poderá dar erros por causa disso.
- O compilador de Java instalado no Mooshak é o openjdk versão 21. Se no vosso desenvolvimento usarem um JDK mais recente, recomendo que configurem o vosso IDE para não usar funcionalidades do Java posteriores à versão que está no Mooshak. (Se usarem o IntelliJ vão a 'Project Structure' → 'Language Level', e escolham 21.

9 Validação e avaliação

Os trabalhos apenas serão validados e avaliados após a discussão individual a ter lugar nas 2 últimas semanas de aulas do semestre. Faz-se notar que os restantes trabalhos serão uma continuação deste.

Não basta terem accepted no mooshak para automaticamente terem boa nota. Aliás, é possível terem boa nota mesmo que não tenham accepted no mooshak.

Os trabalhos serão avaliados de acordo com a clareza e qualidade do código implementado, pela correcta implementação do compilador e máquina virtual, pelo cumprimento dos requisitos acima enunciados, e pelo desempenho individual durante a validação/discussão.

10 Prazo de entrega

O trabalho deve ser realizado em grupo, de acordo com as inscrições em grupo escolhida na tutoria. Deverão submeter o vosso código ao mooshak até às 23:59 do dia 12/Abr/2026. (**NOTA:** Não serão aceites entregas fora de prazo, nem que seja por um só minuto. Não devem deixar a submissão para os últimos instantes. Podem submeter o vosso trabalho várias vezes. Apenas será considerada a última submissão que tiver 'Accepted', ou no caso de não terem Accepted contará a última submissão.